AIMERZ.ai
Aim.Act.Achieve

NOTES

Handling Exceptions in Java -III

Creating Custom Exception

Handling Closable classes using try with resource

Custom Exceptions

Custom exceptions allow creating specific exceptions for unique scenarios by extending the Exception class.

Real-World Analogy: Company Policy

Just as a company might have its own specific rules (like no jeans on Fridays), custom exceptions define specific rules in your program.

Code Syntax

```java
class MyException extends Exception {
    public MyException(String message) {
        super(message);
    }
}
```

Example: Custom Voting Exception

```java
class VotingException extends Exception {
    public VotingException(String message) {
        super(message);
    }
}

public class Voting {
    static void checkVote(int age) throws VotingException {
        if (age < 18) {
            throw new VotingException("Underage! Not allowed to vote.");
        }
    }

    public static void main(String[] args) {
        try {
            checkVote(16);
        } catch (VotingException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Key Differences Between Error and Exception

Aspect	Error	Exception
Severity	Severe issues beyond program control	Less severe, usually related to program logic
Recoverability	Generally unrecoverable; program halts	Often recoverable; can be handled with try-catch
Hierarchy	Subclass of java.lang.Error	Subclass of java.lang.Exception
Examples	OutOfMemoryError, StackOverflowError	IOException, ArithmeticException, NullPointerException
Handling	Usually not handled with try-catch	Can be caught and handled with try-catch
Origin	System-related, resource issues	Runtime issues in the program

When to Handle Errors vs. Exceptions

Errors: Generally, errors should not be handled in the code, as they indicate critical problems with the runtime environment. Handling errors is often impossible or inadvisable, as they represent serious system failures.

Exceptions: Exceptions should be handled using try-catch blocks or declared with throws in the method signature, especially for expected issues like file I/O operations, invalid user inputs, or resource access issues.

Use Cases for Errors and Exceptions in Java

Understanding the scenarios where errors and exceptions occur helps in deciding the best handling approach in each case. Below are some common use cases for both **errors** and **exceptions** in Java:

Use Cases for Errors

Errors in Java usually indicate critical system failures or resource limitations, often outside the program's control. Below are some practical use cases:

OutOfMemoryError

Description: Occurs when the JVM runs out of memory while trying to allocate a new object, often due to memory leaks or loading large amounts of data.

Use Case: Running a data-intensive application that processes very large datasets, like image or video processing.

Handling: Often requires adjusting JVM memory settings (e.g., -Xmx option) or optimizing the code to manage memory better. Rarely handled within the program.

```Java
// Example: Data-intensive operation causing OutOfMemoryError
public class OutOfMemoryExample {
    public static void main(String[] args) {
        // Allocating large array to simulate memory usage
        int[] largeArray = new int[Integer.MAX_VALUE];
    }
}
```

StackOverflowError

Description: Occurs when the call stack is exceeded, often due to deeply nested or infinite recursive calls.

Use Case: Recursive algorithms without a proper base case, such as a recursive function to compute Fibonacci numbers without termination.

Handling: Requires refactoring the code, such as replacing recursion with iterative approaches where possible.

```Java
public class StackOverflowExample {
    public static void recursiveMethod() {
        recursiveMethod(); // Infinite recursion
    }
    public static void main(String[] args) {
        recursiveMethod();
    }
}
```

Use Cases for Exceptions

Exceptions in Java are often more predictable and usually indicate issues with program logic or external resources. Here are common use cases and recommended handling approaches:

IOException

Description: Occurs during input/output operations, such as reading from or writing to files or network streams.

Use Case: Reading a file that does not exist, losing network connectivity while reading a URL, or failing to write to a disk due to permission issues.

Handling: Use try-catch to handle IOException gracefully, such as showing an error message to the user and allowing them to retry or choose another file.

```Java
import java.io.FileReader;
import java.io.IOException;

public class IOExceptionExample {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("non_existent_file.txt");
        } catch (IOException e) {
            System.out.println("File not found. Please check the file
path.");
        }
    }
}
```

NullPointerException

Description: Occurs when the code tries to access a method or field of a null object.

Use Case: Commonly happens when an object has not been initialized but is used. For instance, a method that assumes an object is initialized but receives a null reference.

Handling: Use if checks to validate if an object is null before accessing it or handle it with try-catch when expected.

```java
public class NullPointerExceptionExample {
    public static void main(String[] args) {
        String text = null;
        try {
            System.out.println(text.length()); // Causes
NullPointerException
        } catch (NullPointerException e) {
            System.out.println("Object is null, cannot access its
properties.");
        }
    }
}
```

FileNotFoundException

Description: Occurs when a file specified in the program cannot be found or accessed.

Use Case: When an application attempts to load a configuration file that does not exist or was moved.

Handling: Prompt the user to select a valid file or provide a default configuration. Handle with try-catch and inform the user about the missing file.

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;

public class FileNotFoundExceptionExample {
    public static void main(String[] args) {
        try {
            File file = new File("config.txt");
            FileReader reader = new FileReader(file);
        } catch (FileNotFoundException e) {
            System.out.println("Configuration file not found. Please check
the file path.");
        }
    }
}
```

ArrayIndexOutOfBoundsException

Description: Occurs when attempting to access an array index that is out of bounds.
Use Case: Often occurs in loops when the loop index goes beyond the array size, or when accessing elements of an array without proper bounds checking.
Handling: Use bounds-checking in loops, or handle the exception in try-catch.

```Java
public class ArrayIndexExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};
        try {
            System.out.println(numbers[3]); // Causes
ArrayIndexOutOfBoundsException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of bounds! Please check array
length.");
        }
    }
}
```

Custom Exceptions (User-Defined Exceptions)

Description: Custom exceptions are created to handle specific error conditions unique to the application logic.
Use Case: When implementing domain-specific constraints, such as validating age for voter registration (where age under 18 is invalid).
Handling: Define custom exception classes and use throw to raise exceptions in specific conditions. Handle them using try-catch where applicable.

```Java
class AgeException extends Exception {
    public AgeException(String message) {
        super(message);
    }
}

public class CustomExceptionExample {
    static void checkAge(int age) throws AgeException {
```

```
        if (age < 18) {
            throw new AgeException("Age must be 18 or older to vote.");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(16); // Causes AgeException
        } catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Handling Closable classes using try with resource

Why the Closeable Interface is Used in Java

In Java, the Closeable interface is essential for managing resources that require explicit closing, such as files, network connections, or database connections. It’s part of Java’s way to ensure these resources are closed once they’re no longer needed, which is crucial for efficient memory and resource management.

How Closeable Interface works ?

The Closeable interface provides a single method, close(), which is designed to close a resource when it's no longer in use. When a class implements Closeable, it commits to providing a way to release resources like memory and file handles after use.

Inbuilt class

```
Java
public interface Closeable {
```

```
    void close() throws IOException;
}
```

Purpose of Closeable

Prevent Resource Leaks: Unclosed resources can lead to memory leaks, reduce performance, or even crash the application.

System Resource Management: When files, database connections, or sockets are not closed, they may continue to use system resources (like file descriptors), which are limited.

Automatic Closing: With Closeable, Java can automatically close resources when used with the try-with-resources statement, saving developers from manually managing cleanup.

Real-World Analogy: Library Books

Using a Closeable resource is like borrowing a library book. Once you're done reading, you return it to make it available for others. In Java, the close() method "returns" the resource back to the system, freeing it up for other processes or applications.

Example using BufferedReader which implements Closable Interface

```Java
import java.io.*;

public class Test {
    public static void main(String[] args) throws IOException {

    BufferedReader bf = new BufferedReader(new
InputStreamReader(System.in));

       System.out.println("Enter your Name");
       String name = bf.readLine();
       System.out.println("Your name is "+name);

       bf.close(); // Closing the resource by calling close() method

    }
}
```

Using Scanner to Read Console Input

```Java
import java.util.*;

public class Test {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
        System.out.println("Enter your name");
        String name = input.nextLine();
        System.out.println("Your name is "+name);

        input.close(); // InputResource closed
    }
}
```

What is try-with-resources in Java?

The try-with-resources statement in Java is a special try statement that automatically manages resources (like files, streams, and connections) that must be closed after use.

When a resource is used in a try-with-resources statement, it is closed automatically when the try block completes, regardless of whether an exception occurred. This saves developers from having to manually close each resource.

Why Use try-with-resources?

Automatic Resource Management: Ensures resources are closed automatically, reducing resource leaks.

Simpler Code: Eliminates the need for a finally block to close resources manually.

Enhanced Readability: Clearer and more concise code compared to manual resource management.

Syntax of try-with-resources

```java
try (ResourceType resource = new ResourceType()) {
    // Code that uses the resource
} catch (ExceptionType e) {
    // Code to handle the exception
}
```

In this syntax:

The resource is created within the parentheses () of the try statement.

Java will automatically call the close() method on the resource once the try block is done, whether or not an exception is thrown.

How try-with-resources Works

Resource Declaration: The resource is declared and opened within the try block.

Automatic Closing: When the try block finishes (with or without an exception), the resource's close() method is called automatically, cleaning up the resource.

Real-World Analogy: Borrowing and Returning Tools

Imagine you go to a tool rental shop to borrow a tool. You use it, and when you're done, you automatically return it to avoid a rental fee. In Java, try-with-resources automates the process of "returning" (or closing) the resource, so you don't accidentally forget and incur extra fees (resource leaks).

Detailed Examples and Use Cases for try-with-resources

Reading from a File with BufferedReader

Use Case: When reading an input, it's important to close the input once the input is complete. Using try-with-resources ensures that the file is closed automatically, preventing file lock issues or resource leaks.

```java
import java.io.*;

public class Test {

    public static void main(String[] args) {

        try (BufferedReader bf = new BufferedReader(new
InputStreamReader(System.in))) {
            System.out.println("Enter your Name");
            String name = bf.readLine();
            System.out.println("Your name is " + name);

        } catch (IOException e) {
            e.printStackTrace();
        }
// Here BufferedReader is automatically called after try catch block
    }
}
```

Explanation: Here, BufferedReader reads an input. Using try-with-resources, BufferedReader is automatically closed at the end of the try block, ensuring no open resource are left.

Using Scanner to Read Console Input

Use Case: Using Scanner to read user input from the console, automatically closing the input stream once reading is done.

```java
import java.util.Scanner;

public class ConsoleInputExample {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Enter your name: ");
            String name = scanner.nextLine();
            System.out.println("Hello, " + name + "!");
        }
        // Scanner is automatically closed after the try block.
    }
}
```

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan